

How to Program in Python with Visual Studio Code



Introduction

Visual Studio Code, usually called **VS Code**, is one of the most popular code editors for Python development. It is free, works on Windows, macOS, and Linux, and supports Python through Microsoft's official Python extension. With the right setup, VS Code gives beginners an easy way to write code, run programs, debug mistakes, and install helpful extensions as they grow. (Visual Studio Code)

For a beginner, this matters because learning Python is already a big step. You do not want your tools to feel confusing. VS Code is a strong choice because it can start simple, but it also has room to grow with you as you move from short scripts to larger projects. (Visual Studio Code)

What You Need Before You Start

To program in Python with VS Code, you need two main things:

1. **Visual Studio Code**
2. **A Python interpreter**

VS Code is the editor where you write your code. Python is the software that actually runs your `.py` files. The official VS Code documentation recommends installing both VS Code and a Python interpreter before getting started. ([Visual Studio Code](#))

Step 1 — Install Python

First, install Python from the official Python website. The Python downloads page provides installers for Windows, macOS, and other platforms. ([Python.org](https://python.org))

After installing Python, open a terminal or command prompt and verify that it is installed.

On macOS or Linux:

```
python3 --version
```

On Windows:

```
python --version
```

If Python is installed correctly, you should see a version number.

Step 2 — Install Visual Studio Code

Next, download and install Visual Studio Code from the official VS Code download page. VS Code is available for Windows, macOS, and Linux. ([Visual Studio Code](#))

Step 3 — Install the Python Extension in VS Code

Once VS Code is installed, open it and go to the **Extensions** view. Search for **Python** and install Microsoft's official Python extension. This extension adds rich Python support, including code completion, debugging, linting, refactoring, test support, and environment management. ([Visual Studio Marketplace](#))

A nice benefit for beginners is that the Python extension automatically installs some related tools, including **Pylance** for IntelliSense and language support, and **Python Debugger** for debugging. ([Visual Studio Marketplace](#))

Step 4 — Create Your First Python Program

Create a new file named:

```
hello_world.py
```

Add this code:

```
print(Hello, World!)
```

This is the traditional starting point in almost every programming language. The reason is simple: it is small, easy to understand, and gives you a quick success. It helps confirm that your editor, Python installation, and terminal are all working correctly before you move on to more advanced topics. ([Visual Studio Code](#))

Step 5 — Run the Program

You can run your program from the VS Code terminal.

Open the terminal in VS Code and run:

On macOS or Linux:

```
python3 hello_world.py
```

On Windows:

```
python hello_world.py
```

You should see:



Hello, World!

VS Code also supports running and debugging Python directly from the editor through the Python extension. ([Visual Studio Code](#))

How Beginners Typically Work in VS Code

A very common beginner workflow looks like this:

1. Open a folder for your project in VS Code
2. Create a Python file such as `hello_world.py`
3. Write your code
4. Save the file
5. Run it from the built-in terminal
6. Fix any mistakes and run it again

This simple loop of **write, run, check, improve** is how real programming happens. VS Code supports that cycle very well with syntax highlighting, autocomplete, debugging, and terminal access in one place. (Visual Studio Code)

A Simple Example

Here is a slightly expanded beginner example:



```
name = input(Enter your name: )  
print>Hello,, name)  
print>Welcome to Python programming.)
```

This program introduces two important ideas:

- `input()` lets the user type something
- `print()` displays output on the screen

You can run it the same way:

On macOS or Linux:

```
python3 hello_world.py
```

On Windows:

```
python hello_world.py
```

Example output:

```
Enter your name: Eric  
Hello, Eric  
Welcome to Python programming.
```

Building Your Program Iteratively

One of the best habits for a beginner is to build programs **iteratively**. That means you start with something very small, make sure it works, and then add one new feature at a time.

Version 1



```
print(Hello, World!)
```

Version 2



```
print>Hello, World!  
print>Welcome to Python programming.)
```

Version 3

```
name = input(Enter your name: )  
print>Hello,, name)
```

Version 4

```
name = input(Enter your name: )  
goal = input(What do you want to learn? )  
print>Hello,, name)  
print(It is great that you want to learn, goal)
```

This step-by-step approach helps students avoid frustration. If something breaks, they only need to look at the last small change they made. That is much easier than trying to debug a big program all at once.

Popular Python Extensions for VS Code

As students become more comfortable, they can add more tools to improve their workflow.

1. Python

This is the core extension from Microsoft. It adds Python language support, environment management, debugging access, linting and formatting integration, refactoring, code navigation, and testing features. This is the most important extension for beginners. ([Visual Studio Marketplace](#))

2. Pylance

Pylance provides fast IntelliSense, type information, code completion, and language analysis. For beginners, the biggest value is better autocomplete and clearer code suggestions while typing. ([Visual Studio Marketplace](#))

Use cases:

- Suggesting variable and function names
- Helping students discover methods on strings, lists, and dictionaries
- Catching some mistakes before the program runs

3. Python Debugger

The Python Debugger extension supports breakpoints, stepping through code, and inspecting variables. This is useful when students want to understand what their program is doing line by line. ([Visual Studio Marketplace](#))

Use cases:

- Finding where a program stops working
- Watching variable values change
- Stepping through input and output logic

4. Jupyter

The Jupyter extension adds notebook support inside VS Code. This is especially useful for data analysis, machine learning, and classroom demonstrations. Note that Jupyter is not automatically installed with the Python extension by default anymore. ([Visual Studio Marketplace](#))

Use cases:

- Running code in small cells
- Explaining Python interactively in class
- Data science and beginner experiments

5. Black Formatter

Black Formatter formats Python files automatically. This helps students keep their code neat and consistent. ([Visual Studio Marketplace](#))

Use cases:

- Cleaning up indentation and spacing
- Making code easier to read
- Teaching students that readable code matters

6. Ruff

Ruff is a fast Python linter and formatter extension. It can point out problems in code and help enforce code quality rules. ([Visual Studio Marketplace](#))

Use cases:

- Warning about unused variables
- Catching style issues
- Helping students learn cleaner coding habits

Choosing Extensions as a Beginner

For a true beginner, I would recommend this progression:

Start with:

- **Python**
- **Pylance**

Then add:

- **Python Debugger**
- **Black Formatter**

Later, depending on the course:

- **Jupyter** for notebooks
- **Ruff** for linting and code quality

This keeps the toolset simple at first, which helps beginners focus on learning Python instead of learning too many tools at once.

Helpful Official Links



VS Code Download:

<https://code.visualstudio.com/download>

Python Download:

<https://www.python.org/downloads/>

Python Extension:

<https://marketplace.visualstudio.com/items?itemName=ms-python.python>

Pylance:

<https://marketplace.visualstudio.com/items?itemName=ms-python.vscode-pylance>

Python Debugger:

<https://marketplace.visualstudio.com/items?itemName=ms-python.debugpy>

Jupyter:

<https://marketplace.visualstudio.com/items?itemName=ms-toolsai.jupyter>

Black Formatter:

<https://marketplace.visualstudio.com/items?itemName=ms-python.black-formatter>

Ruff:

<https://marketplace.visualstudio.com/items?itemName=charliermarsh.ruff>

These are all based on the official VS Code or Visual Studio Marketplace pages. (Visual Studio Code)

Tips for Beginner Developers

When first learning Python in VS Code, students should remember a few simple habits:

- Save your file before running it
- Watch the terminal carefully for error messages
- Start with very small programs
- Change one thing at a time
- Use the debugger when a program does not behave as expected

These habits make learning smoother and reduce frustration.

Summary

Visual Studio Code is an excellent place for beginners to learn Python because it combines editing, running, debugging, and extensions in one tool. To get started, students need to install Python, install VS Code, and then install Microsoft's Python extension. From there, they can create a simple file like `hello_world.py`, run it from the terminal, and gradually build more detailed programs one step at a time. (Visual Studio Code)

As students grow, tools like Pylance, Python Debugger, Jupyter, Black Formatter, and Ruff can make development easier and more professional. The key idea is to begin with something simple, get comfortable with the workflow, and then build skill through small, steady improvements. (Visual Studio Marketplace)